total ordering on keys, so we can add and remove elements from the tail of the array, and then bubble up/push down a single element of the heap to reinstate heap's properties (see chapter 2).

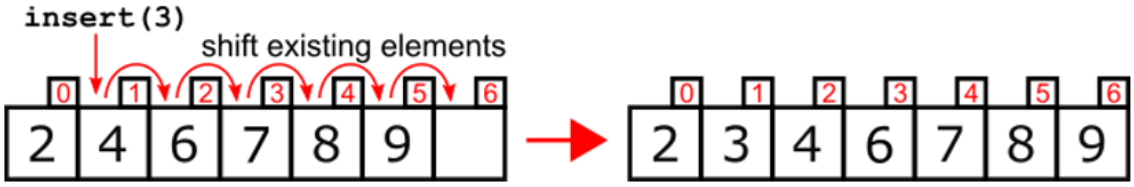


Figure 3.7 When a new element is inserted in an ordered array, all the elements larger than the new one must be shifted one position towards the tail of the array (provided there is still room). This means that, if the array has `n` elements and the new value will be stored at index `k` (i.e., it will be the `(k+1)`-th element in the array), then `n-k` assignments will be needed to complete the insertion.

Treaps, on the other hand, are also binary search trees, which, in turn, do keep a total ordering on keys: that's why we need rotations when we insert or delete new elements form a treap. As we described in section 3.2.1, a rotation implies moving a whole subtree from the right subtree of a node X to the left subtree of its parent Y (or vice versa): as you can imagine, this is the kind of operation that is easily performed in constant time when using pointers on tree's nodes, but it can become excruciatingly painful on arrays (like, linear-time painful).

And that's why the array representation is not used for treaps (nor for BSTs).

Another design question you might ask (and also *should* ask) before getting on with the implementation concerns the branching factor for the heap: we have seen, chapter 2, that heaps can have branching factors other than 2, and in section 2.10 we also saw that a heap with a branching factor of 4 or